

BÀI 07: LỚP TRỪ TƯỢNG VÀ GIAO DIỆN

Mục tiêu

Về kiến thức:

- Giúp sinh viên hiểu được về lớp trừu tượng và giao diện, cùng với mục đích và cách triển khai của chúng..

Về kỹ năng:

Giúp sinh viên phân tích và thiết kế các lớp trừu tượng cho bài toán cụ thể, phân biệt lớp trừu tượng và lớp giao diện, và thực hành viết chương trình để triển khai các lớp này.

7.1. Lớp trừu tượng

Trong lập trình hướng đối tượng, lớp trừu tượng (abstract class) là một lớp không thể tạo đối tượng trực tiếp từ nó, mà phải thông qua các lớp con kế thừa. Lớp trừu tượng đóng vai trò là một lớp cơ sở, định nghĩa các thuộc tính và phương thức mà các lớp con phải triển khai, hoặc có thể đã triển khai sẵn một số phương thức chung.

7.1.1. Đặc điểm của lớp trừu tượng

- Có từ khóa abstract: Một lớp được khai báo là trừu tượng khi có từ khóa abstract trước từ khóa class.
- Không thể khởi tạo đối tượng trực tiếp: Không thể tạo ra đối tượng của lớp trừu tượng bằng từ khóa new.
- Có thể có các phương thức trừu tượng và không trừu tượng:
- Phương thức trừu tượng: chỉ khai báo, không có thân hàm và bắt buộc các lớp con phải định nghĩa lại.
- Phương thức không trừu tượng: phương thức có thân hàm và có thể được kế thừa hoặc ghi đè (override) bởi các lớp con.
- Có thể có thuộc tính và constructor: Mặc dù không thể khởi tạo trực tiếp lớp trừu tượng, nhưng các lớp con kế thừa vẫn có thể sử dụng các thuộc tính và constructor của nó.

6.1.2. Các thành phần của lớp trừu tượng

- Thuộc tính (Fields): Các biến lưu trữ trạng thái của đối tượng.
- Constructor: Lớp trừu tượng có thể có constructor, nhưng không thể khởi tạo đối tượng từ lớp trừu tượng.

- Phương thức trừu tượng (Abstract Method): Phương thức không có thân hàm, chỉ khai báo, và các lớp con phải cài đặt lại.
- Phương thức không trừu tượng (Non-abstract Method): Phương thức đã có cài đặt và có thể được các lớp con kế thừa.

Ví dụ: Giả sử có một bài toán yêu cầu thiết kế hệ thống quản lý động vật trong sở thú, trong đó mỗi loài động vật đều có một hành vi di chuyển riêng.

```
// Lớp trừu tượng
abstract class Animal {
    String name;

    // Constructor của lớp trừu tượng
    public Animal(String name) {
        this.name = name;
    }

    // Phương thức trừu tượng: bắt buộc các lớp con phải triển khai
    public abstract void move();

    // Phương thức không trừu tượng
    public void eat() {
        System.out.println(name + " is eating.");
    }
}
```

7.1.2. Tính chất của lớp trừu tượng

- Tính kế thừa (Inheritance): Các lớp con kế thừa từ lớp trừu tượng và phải triển khai các phương thức trừu tượng.
- Tính đa hình (Polymorphism): Lớp trừu tượng có thể sử dụng để khai báo kiểu dữ liệu, và có thể chứa tham chiếu đến các đối tượng của các lớp con.
- Tính che giấu thông tin (Encapsulation): Lớp trừu tượng giúp che giấu một phần thông tin và chi tiết cài đặt. Các lớp con chỉ cần biết phải triển khai các phương thức đã được khai báo, không cần quan tâm đến cách hoạt động cụ thể của lớp trừu tượng.

Ví dụ: Lớp con Bird

```
// Lớp Bird kế thừa từ Animal và triển khai phương thức move()
class Bird extends Animal {

    public Bird(String name) {
```

```

        super(name);
    }

    @Override
    public void move() {
        System.out.println(name + " is flying.");
    }
}

```

Lớp con Fish

```

// Lớp Fish kế thừa từ Animal và triển khai phương thức move()
class Fish extends Animal {

    public Fish(String name) {
        super(name);
    }

    @Override
    public void move() {
        System.out.println(name + " is swimming.");
    }
}

```

Chương trình chính

```

public class Zoo {
    public static void main(String[] args) {
        Animal bird = new Bird("Parrot");
        Animal fish = new Fish("Goldfish");

        bird.move(); // Output: Parrot is flying.
        bird.eat(); // Output: Parrot is eating.

        fish.move(); // Output: Goldfish is swimming.
        fish.eat(); // Output: Goldfish is eating.
    }
}

```

Lớp *Animal* là lớp trừu tượng, có phương thức trừu tượng *move()* và phương thức không trừu tượng *eat()*.

Lớp *Bird* và *Fish* kế thừa từ *Animal* và triển khai phương thức *move()*.

Chương trình chính tạo đối tượng của *Bird* và *Fish*, rồi gọi các phương thức *move()* và *eat()* để hiển thị hành vi di chuyển và ăn uống của mỗi loài động vật.

Lớp trừu tượng giúp chúng ta tổ chức và quản lý mã nguồn một cách khoa học, đảm bảo tính kế thừa và sự linh hoạt trong việc mở rộng hệ thống.

7.2. Giao diện (Interface)

Trong lập trình hướng đối tượng, Interface là một cơ chế được sử dụng để định nghĩa các hành vi (methods) mà các lớp (classes) phải tuân thủ. Một interface chỉ chứa khai báo của các phương thức, không chứa định nghĩa (implementation). Các lớp khác có thể implements (triển khai) interface bằng cách cung cấp các định nghĩa cho các phương thức đã khai báo trong interface đó.

Một Interface trong Java là một bản thiết kế của một lớp. Nó chỉ có các phương thức trừu tượng. Interface là một kỹ thuật để thu được tính trừu tượng hoàn toàn và đa kế thừa trong Java. Interface trong Java cũng biểu diễn mối quan hệ IS-A. Nó không thể được khởi tạo giống như lớp trừu tượng. Nói cách khác, các thành phần của Interface là public, static và final theo mặc định và các phương thức là public và abstract.

Một Interface trong Java là một tập hợp các phương thức trừu tượng (abstract). Một class triển khai một interface, do đó kế thừa các phương thức abstract của interface.

Một interface không phải là một lớp. Viết một interface giống như viết một lớp, nhưng chúng có 2 định nghĩa khác nhau. Một lớp mô tả các thuộc tính và hành vi của một đối tượng. Một interface chứa các hành vi mà một class triển khai. Trừ khi một lớp triển khai interface là lớp trừu tượng abstract, còn lại tất cả các phương thức của interface cần được định nghĩa trong class.

7.2.1. Đặc điểm của Interface

Một interface tương tự với một class bởi những điểm sau đây:

- Một interface được viết trong một file với định dạng .java, với tên của interface giống tên của file.
- Bytecode của interface được lưu trong file có định dạng .class.
- Khai báo interface trong một package, những file bytecode tương ứng cũng có cấu trúc thư mục có cùng tên package.

Một interface khác với một class ở một số điểm sau đây:

- Interface không có thể hiện (instance) do đó một interface không chứa bất cứ hàm Constructor nào.
- Tất cả các phương thức của interface đều là abstract.

- Trong interface chỉ có các thuộc tính static final
- Một interface không thể kế thừa từ lớp, nó được triển khai bởi một lớp.
- Một interface có thể kế thừa từ nhiều interface khác.

Các bước để tạo một interface:

- Định nghĩa Interface
- Biên dịch Interface
- Hiện thực hay dẫn xuất (implements) Interface

Tính chất của interface:

- Tất cả phương thức trong interface phải là public.
- Các phương thức phải được định nghĩa trong lớp dẫn xuất giao diện đó.
- Không thể dẫn xuất từ lớp khác, nhưng có thể dẫn xuất từ những interface khác

Khi định nghĩa một interface mới thì một kiểu dữ liệu tham chiếu cũng được tạo ra.

7.2.2. Khai báo interface

Cú pháp khai báo một interface như sau:

```
[public] interface <tên interface> [extends <danh sách interface>]
{ ... }
```

Các thành phần của Interface

Khai báo phương thức của interface

```
[public] <kiểu giá trị trả về> <tên phương thức> ([<các tham số>])
[throws <danh sách ngoại lệ>];
```

Tính chất: tính chất của một thuộc tính hay phương thức của *interface* luôn là *public*. Nếu không khai báo tường minh thì giá trị mặc định cũng là *public*.

Kiểu giá trị trả về: có thể là các kiểu cơ bản của java, cũng có thể là kiểu do người dùng tự định nghĩa (kiểu đối tượng).

Tên phương thức: tuân thủ theo quy tắc đặt tên phương thức của lớp

Các tham số: nếu có thì mỗi tham số được xác định bằng một cặp <kiểu tham số> <tên tham số>. Các tham số được phân cách nhau bởi dấu phẩy.

Các ngoại lệ: nếu có thì mỗi ngoại lệ được phân cách nhau bởi dấu phẩy.

Lưu ý:

- Các phương thức của *interface* chỉ được khai báo dưới dạng mẫu mà không có cài đặt chi tiết (có dấu chấm phẩy ngay sau khai báo và không có phần cài đặt trong dấu “{}”). Phần cài đặt chi tiết của các phương thức chỉ được thực hiện trong các lớp (*class*) sử dụng *interface* đó.
- Các thuộc tính của *interface* luôn có tính chất là hằng (*final*), tĩnh (*static*) và *public*. Do đó, cần gán giá trị khởi đầu ngay khi khai báo thuộc tính của *interface*.

Khai báo phương thức trừu tượng: Đây là các phương thức không có phần thân. Ví dụ: **public void** draw();

Hằng số: Các biến trong interface mặc định là public static final, có nghĩa là chúng là các hằng số. Ví dụ: int MAX_WIDTH = 1024;

Phương thức mặc định (default methods): Phương thức có thể có phần thân trong interface từ Java 8 trở đi.

```
default void log(String message) {
    System.out.println("Log: " + message);
}
```

Phương thức tĩnh (static methods): Từ Java 8, interface có thể có phương thức tĩnh.

```
static void printVersion() {
    System.out.println("Interface version 1.0");
}
```

Ví dụ minh họa: Thiết kế một hệ thống cho phép vẽ các hình học khác nhau như hình tròn, hình chữ nhật và tam giác. Mỗi hình sẽ có một phương thức draw() để vẽ hình đó.

Interface cho bài toán:

```
// Interface Shape chứa phương thức draw
public interface Shape {
    void draw();
}
```

Lớp triển khai cho hình tròn:

```
// Lớp Circle triển khai interface Shape
public class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a Circle");
    }
}
```

Lớp triển khai cho hình chữ nhật:

```
// Lớp Rectangle triển khai interface Shape
public class Rectangle implements Shape {
```

```

@Override
public void draw() {
    System.out.println("Drawing a Rectangle");
}
}

```

Lớp triển khai cho hình tam giác:

```

// Lớp Triangle triển khai interface Shape
public class Triangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a Triangle");
    }
}

```

Lớp chính:

```

public class Main {
    public static void main(String[] args) {
        // Tạo các đối tượng hình học
        Shape circle = new Circle();
        Shape rectangle = new Rectangle();
        Shape triangle = new Triangle();

        // Gọi phương thức draw cho mỗi hình
        circle.draw();
        rectangle.draw();
        triangle.draw();
    }
}

```

Kết quả đầu ra:

```

Drawing a Circle
Drawing a Rectangle
Drawing a Triangle

```

- **Interface Shape** chỉ chứa phương thức trừu tượng draw(). Các lớp Circle, Rectangle, và Triangle đều triển khai phương thức này theo cách riêng của chúng.
- **Tính đa hình** được thể hiện rõ khi chúng ta có thể sử dụng cùng một kiểu Shape cho các đối tượng khác nhau (Circle, Rectangle, Triangle) và vẫn có thể gọi phương thức draw() trên chúng mà không cần quan tâm đến lớp cụ thể của đối tượng đó.

Interface giúp xây dựng hệ thống linh hoạt và dễ mở rộng, nơi chúng ta có thể dễ dàng thêm các hình mới mà không ảnh hưởng đến các phần khác của hệ thống.

7.2.3. Quan hệ kế thừa giữa các giao diện

Trong Java, một lớp có thể **kế thừa** một lớp khác bằng từ khóa `extends`. Lớp con kế thừa các thuộc tính và phương thức của lớp cha và có thể mở rộng hoặc ghi đè (override) chúng. Ví dụ:

```
class Animal {  
    void sound() {  
        System.out.println("Some sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Barking");  
    }  
}
```

Ở đây, lớp `Dog` kế thừa lớp `Animal` và ghi đè phương thức `sound()`.

Đặc điểm của kế thừa lớp:

- Một lớp chỉ có thể kế thừa **một lớp cha** (Java không hỗ trợ đa kế thừa từ nhiều lớp).
- Lớp con có thể ghi đè (override) các phương thức của lớp cha để cung cấp hành vi riêng.
- Các phương thức hoặc thuộc tính `private` của lớp cha không thể truy cập trực tiếp từ lớp con.
- Kế thừa hỗ trợ tính mở rộng, giúp tái sử dụng mã nguồn và bổ sung tính năng mà không cần thay đổi lớp cha.

Quan hệ kế thừa giữa các giao diện

Một giao diện có thể **kế thừa** từ một hoặc nhiều giao diện khác bằng từ khóa `extends`. Khi một giao diện kế thừa một giao diện khác, nó có thể thêm các phương thức mới vào danh sách các phương thức trừu tượng mà nó yêu cầu các lớp triển khai phải tuân thủ. Ví dụ:

```
interface Animal {  
    void sound();  
}  
  
interface Pet extends Animal {  
    void play();  
}
```

Trong ví dụ trên, giao diện Pet kế thừa từ giao diện Animal. Một lớp triển khai giao diện Pet phải cung cấp phần thực thi cho cả hai phương thức sound() (từ Animal) và play() (từ Pet).

Đặc điểm của kế thừa giao diện:

- Một giao diện có thể kế thừa từ nhiều giao diện khác (đa kế thừa).
- Giao diện con kế thừa tất cả các phương thức trừu tượng từ giao diện cha.
- Các phương thức mặc định (default methods) và phương thức tĩnh (static methods) từ Java 8 trở đi cũng có thể được kế thừa và sử dụng.

7.2.4. Mở rộng quan hệ kế thừa bằng cách triển khai nhiều giao diện

Trong Java, một lớp có thể **triển khai** nhiều giao diện (multi-implements), điều này cung cấp cho lập trình viên khả năng linh hoạt trong việc thiết kế hệ thống mà không gặp phải vấn đề của đa kế thừa từ nhiều lớp. Khi một lớp triển khai nhiều giao diện, nó phải cung cấp phần thực thi cho tất cả các phương thức được khai báo trong các giao diện đó. Ví dụ:

```
interface Movable {  
    void move();  
}  
  
interface Flyable {  
    void fly();  
}  
  
class Airplane implements Movable, Flyable {  
    @Override  
    public void move() {  
        System.out.println("Airplane is moving");  
    }  
  
    @Override  
    public void fly() {  
        System.out.println("Airplane is flying");  
    }  
}
```

Trong ví dụ này, lớp Airplane triển khai cả hai giao diện Movable và Flyable. Điều này giúp lớp Airplane có thể đảm nhận cả hành vi "di chuyển" và "bay".

Đặc điểm của triển khai nhiều giao diện:

- Một lớp có thể triển khai nhiều giao diện cùng lúc bằng cách sử dụng từ khóa implements.

- Nếu có nhiều giao diện có các phương thức trùng tên, lớp triển khai phải định nghĩa tất cả các phương thức đó, và có thể phân biệt hành vi cho từng giao diện.
- Việc triển khai nhiều giao diện giúp tạo ra các đối tượng có nhiều hành vi đa dạng mà không cần kế thừa từ nhiều lớp (không gặp vấn đề của đa kế thừa từ nhiều lớp như trong C++).

7.2.5. Sự kết hợp giữa kế thừa và giao diện trong Java

Trong Java, có thể kết hợp cả kế thừa lớp và triển khai giao diện trong một lớp. Một lớp có thể kế thừa từ một lớp cha và đồng thời triển khai một hoặc nhiều giao diện.

Ví dụ:

```
class Vehicle {
    void start() {
        System.out.println("Vehicle is starting");
    }
}

interface Drivable {
    void drive();
}

interface Flyable {
    void fly();
}

class FlyingCar extends Vehicle implements Drivable, Flyable {
    @Override
    public void drive() {
        System.out.println("FlyingCar is driving");
    }

    @Override
    public void fly() {
        System.out.println("FlyingCar is flying");
    }
}
```

Trong ví dụ này, lớp FlyingCar kế thừa từ lớp Vehicle và đồng thời triển khai hai giao diện Drivable và Flyable. Lớp FlyingCar có thể thừa hưởng phương thức start() từ Vehicle và cung cấp phần thực thi cho hai phương thức drive() và fly() từ các giao diện.

Ưu điểm của sự kết hợp giữa kế thừa và giao diện:

- **Tính mở rộng linh hoạt:** Bằng cách sử dụng giao diện kết hợp với kế thừa lớp, ta có thể tạo ra các đối tượng với nhiều hành vi khác nhau.
- **Hỗ trợ đa kế thừa từ giao diện:** Mặc dù Java không hỗ trợ đa kế thừa từ nhiều lớp, nhưng có thể sử dụng giao diện để đạt được mục tiêu này.
- **Tái sử dụng mã nguồn:** Kế thừa từ các lớp cha giúp tái sử dụng các phương thức và thuộc tính sẵn có, trong khi giao diện giúp đảm bảo rằng các lớp triển khai tuân thủ các hành vi được định nghĩa sẵn.

Quan hệ kế thừa và giao diện trong Java là những yếu tố quan trọng giúp tạo nên tính đa hình, tính mở rộng, và tính linh hoạt trong thiết kế phần mềm. Trong khi lớp chỉ có thể kế thừa một lớp cha, Java bù đắp điều này bằng khả năng triển khai nhiều giao diện, cho phép các lớp có thể hỗ trợ nhiều hành vi cùng lúc mà không gây xung đột trong cấu trúc của chương trình.

7.3. Phân biệt lớp trừu tượng và giao diện

Trong lập trình hướng đối tượng, **lớp trừu tượng (abstract class)** và **giao diện (interface)** đều là những công cụ dùng để định nghĩa các hành vi mà các lớp con hoặc lớp triển khai phải tuân theo. Tuy nhiên, chúng có những điểm giống và khác nhau nhất định về cấu trúc, tính năng, và cách sử dụng.

Giống nhau

- **Đều là bản thiết kế:** Cả lớp trừu tượng và giao diện đều không thể tạo đối tượng trực tiếp từ chúng. Chúng đóng vai trò là bản thiết kế để các lớp con hoặc lớp triển khai thực thi các phương thức mà chúng định nghĩa.
- **Hỗ trợ tính đa hình:** Cả hai đều có thể được sử dụng để tạo ra tính đa hình. Các lớp có thể kế thừa lớp trừu tượng hoặc triển khai giao diện, và các đối tượng có thể được thao tác thông qua kiểu lớp trừu tượng hoặc giao diện.
- **Phương thức trừu tượng:** Cả lớp trừu tượng và giao diện đều có thể chứa phương thức trừu tượng (abstract method), yêu cầu các lớp con hoặc lớp triển khai phải định nghĩa.

Khác nhau

	Lớp trừu tượng (Abstract Class)	Giao diện (Interface)
Từ khóa sử dụng	abstract	interface

Khả năng chứa phương thức đã triển khai	Lớp trừu tượng có thể chứa phương thức đã triển khai (có thân phương thức).	Từ Java 8 trở đi, giao diện có thể chứa phương thức mặc định (default) và phương thức tĩnh (static) với phần triển khai. Trước Java 8, tất cả phương thức trong giao diện đều phải là phương thức trừu tượng.
Kế thừa và triển khai	Một lớp chỉ có thể kế thừa từ một lớp trừu tượng. Lớp con phải sử dụng từ khóa extends .	Một lớp có thể triển khai nhiều giao diện cùng lúc. Lớp triển khai phải sử dụng từ khóa implements .
Phương thức trừu tượng	Có thể chứa cả phương thức trừu tượng và phương thức không trừu tượng.	Chỉ chứa phương thức trừu tượng (trước Java 8). Từ Java 8 có thể có thêm phương thức mặc định và phương thức tĩnh.
Biến và hằng số	Lớp trừu tượng có thể có các biến instance (biến đối tượng) và các hằng số.	Trong giao diện, tất cả các biến đều là public static final (hằng số).
Truy cập mặc định của phương thức	Các phương thức trong lớp trừu tượng có thể có các mức độ truy cập khác nhau (public, protected, private).	Các phương thức trong giao diện mặc định là public (không thể có private hoặc protected trừ khi là phương thức tĩnh hoặc phương thức mặc định từ Java 9).
Tính đa kế thừa	Không hỗ trợ đa kế thừa từ nhiều lớp trừu tượng (một lớp chỉ có thể kế thừa từ một lớp trừu tượng).	Hỗ trợ đa kế thừa từ nhiều giao diện (một lớp có thể triển khai nhiều giao diện).
Khi nào nên sử dụng	Dùng khi bạn cần một lớp cơ sở với một số phương thức đã triển khai hoặc có trạng thái (biến instance).	Dùng khi bạn cần định nghĩa các hành vi mà nhiều lớp không liên quan đến nhau có thể triển khai.

7.4. Bài tập mẫu

Bài 1: Tạo 1 lớp trừu tượng (abstract) Person chứa các thông tin về người, tạo tiếp 2 lớp Students và Teachers kế thừa lớp Person, tạo lớp Execute chứa hàm main để chạy chương trình.

```
import java.io.Console;
import java.util.Scanner;
abstract class Person {
```

```

// cac property hay state-thuoc tinh cua doi tuong
String hoten;
int age;
String diachi;
int luong;
//cac constructor
Person() {
}
public Person(int age) {
    this.age = age;
}
//cac method hay behavior-hanh vi cua doi tuong
public void Nhap() {
    Console input = System.console();
    hoten = input.readLine("Nhap ho ten:");
    diachi = input.readLine("Nhap dia chi:");
}
public abstract void In();
public abstract int Tinhluong();
}
class Students extends Person {
    int MaSV, Malop;
    public Students(int age) {
        super(age);
    }
    public Students() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma SV: ");
        MaSV = input.nextInt();
        System.out.println("Nhap Ma Lop: ");
        Malop = input.nextInt();
    }

    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
        System.out.println(MaSV);
        System.out.println(Malop);
    }
    public int Tinhluong() {
        return 150000;
    }
}
class Teachers extends Person {
    int Makhoa;

```

```

    public Teachers(int age) {
        super(age);
    }
    public Teachers() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma khoa: ");
        Makhoa = input.nextInt();
    }
    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
        System.out.println(Makhoa);
    }
    public int Tinhluong() {
        return 500000;
    }
}

public class Execute {
    public static void main(String args[]) {
        Students st = new Students();
        st.Nhap();
        st.In();
        st.luong = st.Tinhluong();
        Teachers tc = new Teachers();
        tc.Nhap();
        tc.In();
        tc.luong = tc.Tinhluong();
    }
}

```

Bài 2: Shape là lớp trừu tượng, trình triển khai của nó được cung cấp bởi lớp **Rectangle** và lớp **Circle**. Hai lớp này kế thừa lớp trừu tượng **Shape**.

```

// lop truu tuong Shape
abstract class Shape {
    abstract void draw();
}

//Trong tinh huong nay, trinh trien khai duoc cung cap boi ai do, vi du: nguoi su
dung cuoi cung nao do
class Rectangle extends Shape {
    void draw() {
        System.out.println("Ve hình chu nhật");
    }
}

class Circle1 extends Shape {
    void draw() {

```

```

        System.out.println("Ve hình tron");
    }
}
//Trong tình huống này, phương thức được gọi bởi lập trình viên hoặc người
dùng
class TestAbstraction1 {
    public static void main(String args[]) {
        Shape s = new Circle1(); //Trong tình huống này, đối tượng được
cung cấp thông qua phương thức, chẳng hạn như getShape()
        s.draw();
    }
}

```

7.5. Bài tập thực hành

Bài 1: Bài toán về các loại động vật và âm thanh

Bài toán: Xây dựng một hệ thống mô tả các loài động vật khác nhau. Mỗi loài động vật có khả năng phát ra âm thanh riêng. Ngoài ra, có một số loài động vật là thú cưng và chúng có thể chơi đùa.

Cách triển khai:

- Sử dụng lớp trừu tượng Animal để định nghĩa phương thức sound().
- Sử dụng giao diện Pet để định nghĩa phương thức play().
- Tạo các lớp cụ thể như Dog, Cat kế thừa từ lớp trừu tượng Animal và triển khai giao diện Pet.

Bài 2: Bài toán về các loại phương tiện giao thông

Bài toán: Xây dựng một hệ thống mô tả các phương tiện giao thông như ô tô, xe máy, và máy bay. Một số phương tiện có thể bay và di chuyển.

Cách triển khai:

- Sử dụng giao diện Movable để định nghĩa phương thức move().
- Sử dụng giao diện Flyable để định nghĩa phương thức fly().
- Sử dụng các lớp như Car, Motorcycle, Airplane để triển khai các giao diện này

Bài 3: Bài toán về các loại tài khoản ngân hàng

Bài toán: Mô phỏng hệ thống tài khoản ngân hàng, với các loại tài khoản như tài khoản tiết kiệm và tài khoản vãng lai. Mỗi loại tài khoản có cách tính lãi khác nhau.

Cách triển khai:

- Sử dụng lớp trừu tượng BankAccount để định nghĩa phương thức calculateInterest().
- Tạo các lớp SavingAccount và CurrentAccount kế thừa từ lớp trừu tượng BankAccount để triển khai cách tính lãi.

Bài 4: Bài toán về thiết kế đồ họa các hình học

Bài toán: Xây dựng hệ thống vẽ các hình học như hình tròn, hình chữ nhật và hình tam giác. Mỗi hình có diện tích và chu vi được tính toán khác nhau.

Cách triển khai:

- Sử dụng lớp trừu tượng Shape để định nghĩa phương thức trừu tượng area() và perimeter().
- Tạo các lớp Circle, Rectangle, và Triangle kế thừa từ Shape và định nghĩa cách tính diện tích, chu vi cho từng loại hình.

Bài 5: Bài toán về hệ thống thiết bị điện tử

Bài toán: Xây dựng hệ thống mô tả các thiết bị điện tử như điện thoại và máy tính xách tay. Một số thiết bị có thể được sạc pin và bật tắt.

Cách triển khai:

- Sử dụng giao diện Chargeable để định nghĩa phương thức charge().
- Sử dụng giao diện Switchable để định nghĩa phương thức turnOn() và turnOff().
- Tạo các lớp Phone và Laptop để triển khai các giao diện này.